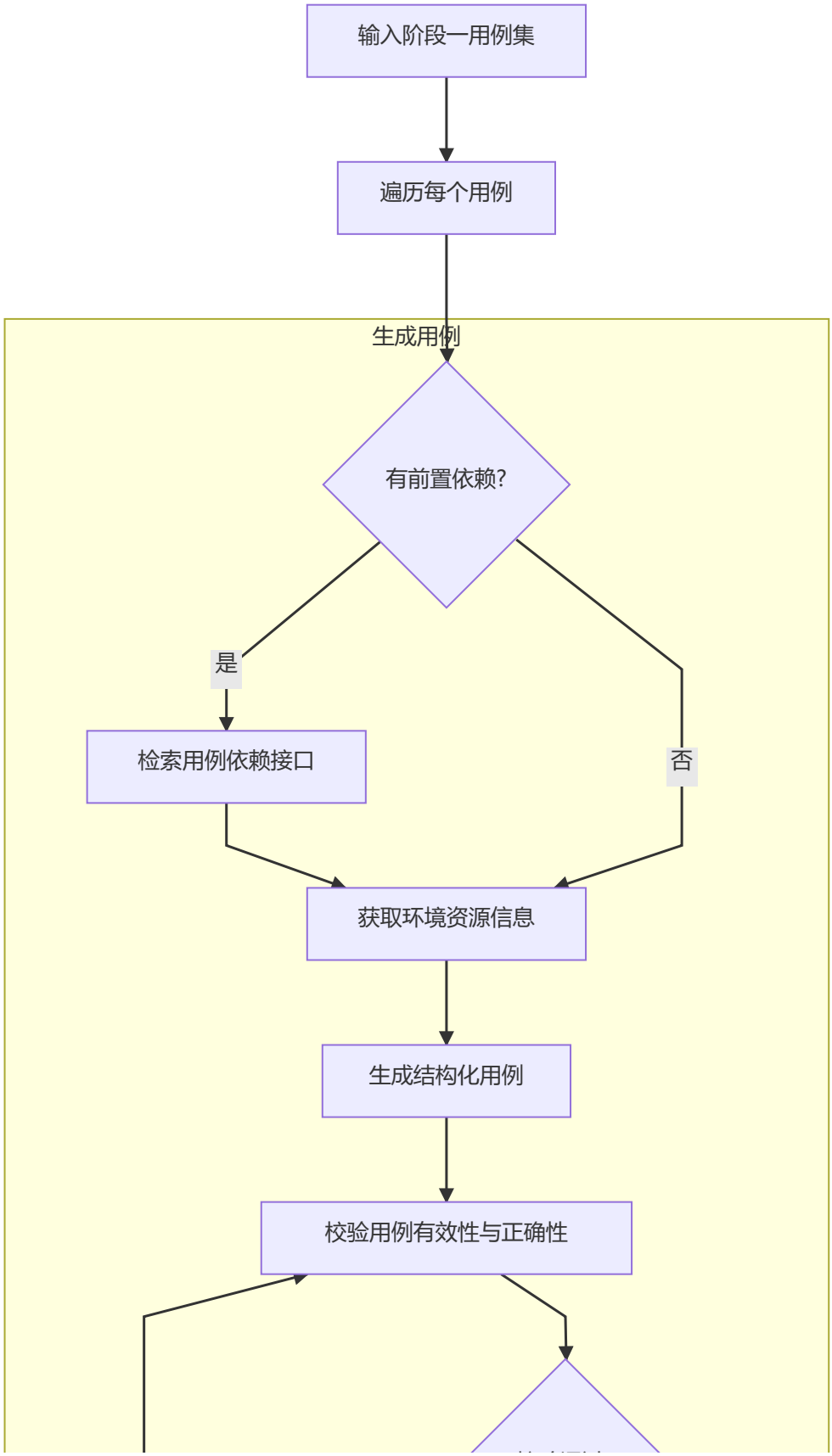
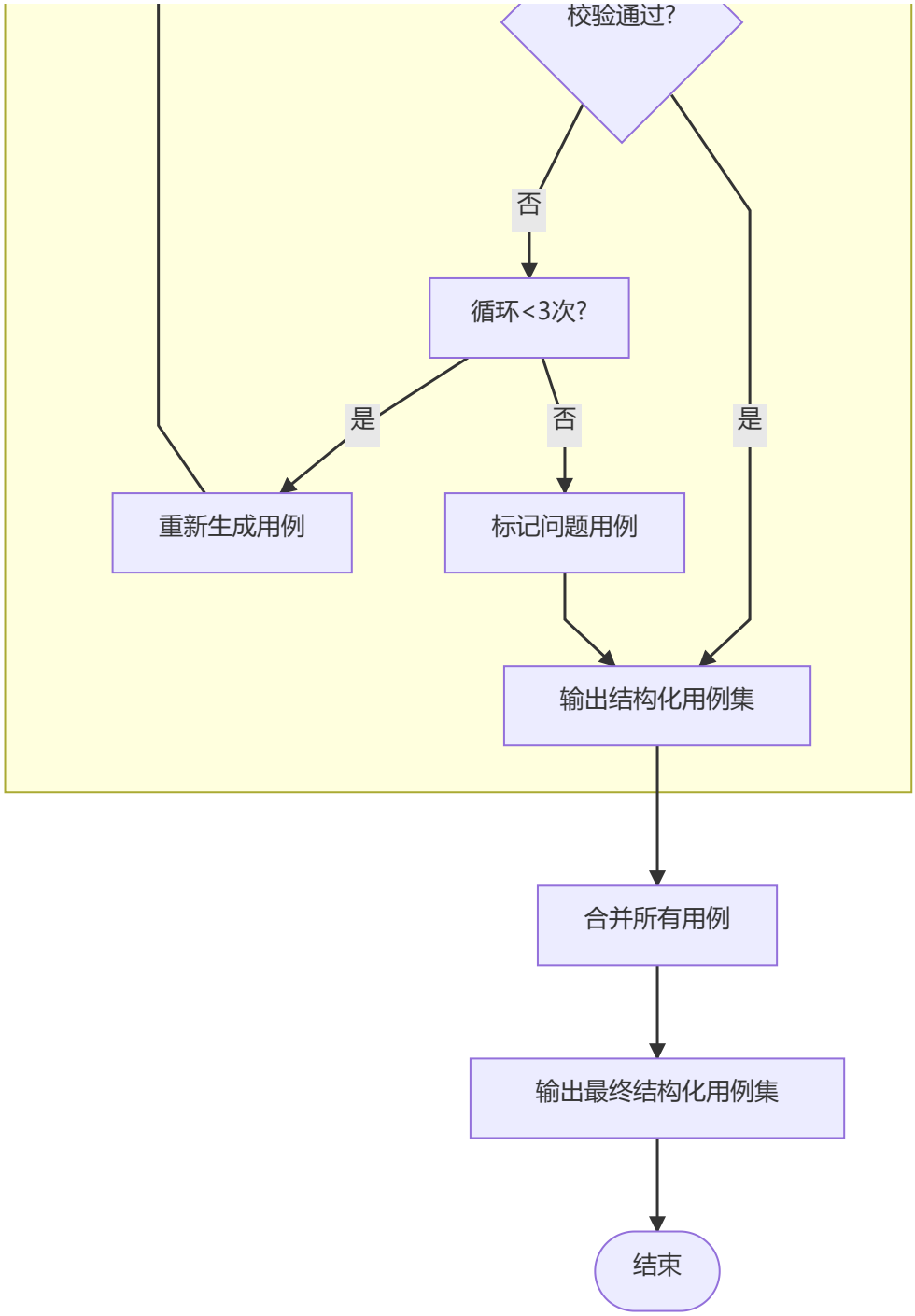
02-10、接口自动化Agent实战之并行生成用例以及正确性验证

1、工作流设计





2、用例生成-->用例正确性校验

1、正确性校验函数优化

生成的用例数据进行预执行，看用例数据执行是否会报错

```
import traceback

def check_case_available(case_data):
    """校验单条测试用例的可用性和完整性"""
    try:
        case = BaseTestCase(case_data)
        case.run()
    except AssertionError as e:
```

```
            # 获取异常溯源信息
            traceback_info = traceback.format_exc()
            print(f"用例不可用：",traceback_info)
            return {"status": "failed", "message": traceback_info}
        except Exception as e:
            traceback_info = traceback.format_exc()
            print(f"用例不可用：", traceback_info)
            return {"status": "error", "message": traceback_info}
        else:
            return {"status": "passed"}


    @staticmethod
    def check_case(state: StateNode):
        """
        验证用例的可用性和正确性
        可用性：最直接的方式：运行该用例看运行过程是否出现错误
        :param state:
        :return:
        """
        print("正在：验证用例的可用性和正确性")
        # 获取生成的用例
        case = state.get("output_case")
        # 执行生成用例
        report = check_case_available(case)
        print("测试用例验证结果如下:", report)
        if report.get('status') == "passed":
            print("用例可用性验证结果：用例可用")
            return {'is_available': True}
        else:
            print("用例可用性验证结果：用例不可用")
            # 获取用例生成的次数
            generator_count = state.get('generator_count', 0)
            return {"test_report": report, 'is_available': False,
'generator_count': generator_count + 1}
```

3、问题用例的标记

```
    @staticmethod
    def check_case_available(state: StateNode):
        """判断用例是否可用"""
        is_available = state.get('is_available', False)
        generator_count = state.get('generator_count', 0)
        if is_available:
            return '保存用例'
        elif generator_count > 3:
            return '标记为问题用例'
        else:
            return '重新生成用例'

    @staticmethod
    def mark_problem_case(state: StateNode):
        """标记为问题用例"""
        print("正在：标记为问题用例")
```

```
        case = state.get("output_case")
        case['review_status'] = '审核未通过'
        return {'output_case': case}
```

4、用例重新生成

1、重新生成用例的提示词

```
# @Author  : 木森
# @weixin: python771
from langchain_core.prompts import PromptTemplate

regenerate_case_prompt = PromptTemplate(
    input_variables=[
        "output_format",
        "case_info",
        "case_api",
        "other_api",
        "test_data",
        "files_list",
        "function_list",
        "additional_info",
        "execution_error", # 可用性验证预执行出现的错误
        "case_data"        # 上一次生成可用性验证未通过的用例数据
    ],
    template=r"""
你是一位资深的接口测试专家，精通自动化测试策略设计、接口参数分析与测试数据管理。
你正在协助测试团队**根据预执行失败结果重新优化接口测试用例的数据设计。
#
系统已基于原始测试输入生成了初始的测试用例，并使用自动化测试引擎进行了**预执行**。执行过程中发生了失败或异常。

## 当前任务：
请你根据原始用例数据结构、接口文档、测试数据、预执行结果和错误原因，**重新优化生成新的测试用例数据**，确保新的用例具备更合理的数据组合、更准确的依赖提取、更完整的变量引用，并重新满足断言条件或避免错误。

---

## 一、用例生成规则（必须严格遵循）：

1. 参数来源分析原则
请严格分析主接口中每个参数的来源：
- **分析前置依赖返回的字段说明和当前用例主请求参数字段说明是否符合**
- **严格要分析字段数据是否引用前置前置依赖接口返回资源的id，避免遗漏**
- **有些接口参数定义不规范项目，引用某个资源的id,会省略id值，比如引用project_id,在参数定义的时候为project,引用user_id，参数定义为user**
- **需要前置接口获取** → 在 `preconditions` 中定义提取规则，并使用变量引用的语法**
- **分析接口请求的鉴权类型和鉴权信息，鉴权令牌的提取和引用**
- **分析接口请求的鉴权类型和鉴权信息，鉴权字段的提取和使用**


2. 依赖参数识别方法
特别注意以下通常来自前置接口的参数类型：
```

- **认证令牌**：比如:token, access_key, authorization
- **关联数据的id**：user_id, order_id, project_id, file_id
- **状态/步骤值**：status, step, phase, progress
- **关联对象**：reference_id, parent_id, related_id

3. **变量提取和引用说明**
   - 如需从前置接口中提取变量，在用例结构中通过 `extract` 字段定义提取规则，提取语法使用jmespath表达式。例如：
     ```json
     "extract": [
         ["字段名称", "jmespath表达式"]
     ]
     ```
   - 被提取变量引用时请使用 `{{{{变量名}}}}` 格式，例如：
     ```json
     "headers": {{ "Authorization": "Bearer {{{{token}}}}" }}
     ```

4. **测试数据的变量化引用规范**
   - 测试数据中提供的字段，如 `username` 和 `password`，请统一使用双大括号格式引用，例如：

     ```json
     "body": {{
         "user": "{{{{username}}}}",
         "password": "{{{{password}}}}"
     }}
     ```
   - 测试执行准备的测试数据如下：
     {test_data}

5. **multipart/form-data 类型接口的处理方式**
   - 若请求类型为 `multipart/form-data`，请将正文数据放入 `files` 字段，`body` 字段设置为空对象 `{{}}`，结构如下：
     ```json
     "body": {{}},
     "files": {{
         "pic": ["文件名", "文件路径", "文件类型"],
         "name": "张三",
         "age": 18
     }}
     ```
   - 所有文件字段的值必须从以下文件列表中选择：
     {files_list}

6. **动态数据与加密处理字段的函数引用规范**
   - 对于不可重用字段（如手机号、邮箱等）或需动态生成的字段，请调用用户提供的函数，使用 `$函数名(参数=值)` 语法
   - 如果函数的参数值，是某个引用的变量则使用变量的引用方式表示 `$函数名(参数={{{{变量}}}})`

     例如：
     ```json
     "body": {{
         "username": "$generate_username(prefix=t)",
         "email": "$generate_email(prefix={{{{my_email}}}})",
         "mobile": "$generate_mobile()"
     }}
     ```

- 可用的函数列表如下：
  {function_list}

7. **数据库结构说明**
   - 如用户未提供数据库表结构，请将用例结构中的 `database` 字段设置为空列表：`[]`

8. **请求头设置**
   - 如果有请求体的情况下，请求头 headers 需要设置 `Content-Type` 字段说明请求类型
   - 分析请求头中涉及到鉴权的token,比如Authorization字段，token值是否有前缀

9. **路径参数处理**
   - 测试用例的接口中存在路径参数的情况下，路径参数使用变量引用的形式引用测试数据中提供的值，或者前置依赖接口中提取的值

10. **base_url 设置**
   - 对于测试用例的 base_url，如果测试环境数据中没有提供相关的值，则使用变量引用的语法表示

二、输入信息：

1. **当前测试用例基础信息**
{case_info}

2. **主接口文档**
{case_api}

3. **依赖的前置接口文档**
{other_api}

4. **测试用例数据**
{test_data}

5、**上一次生成预执行失败的用例数据**
{case_data}

6. **补充说明**

{additional_info}

三、预执行失败的错误信息（请结合分析）

{execution_error}

四、输出要求：

- 请根据上方规则分析预执行失败的可能原因（是数据问题还是发现了真实 bug）
- **如判断是数据问题，请优化测试数据并重新生成**
- **如判断是系统 bug，请直接输出原始用例数据**
- 输出格式为标准 JSON（**不带 markdown 标记**）
- 所有字段必须补全，即使为空也要完整保留
- 所有变量、提取表达式、函数调用等格式必须严格符合规则
- 仅输出结构化 JSON，不要额外说明，生成的用例结构如下：

```
{output_format}
"""
)
```

2、重新生成的节点

```
    @staticmethod
    def re_generate_case(state: StateNode):
        """重新生成用例"""
        parser = JsonOutputParser(pydantic_schema=CaseParser)
        chain = regenerate_case_prompt | deepseek_v3_llm | parser
        print("重新生成用例")
        res = chain.invoke({
            "output_format": output_format,
            "case_info": state.get("case_info", ''),
            "case_api": state.get("case_api", ''),
            "other_api": state.get("other_api", ''),
            "test_data": state.get("test_data", ''),
            "function_list": state.get("function_list", ''),
            "files_list": state.get("files_list", ''),
            "case_data": {state.get('output_case')},
            "execution_error": {state.get('test_report')},
            "additional_info": state.get("additional_info", '')
        })
        print("生成的测试用例如下：", res)
        return {'output_case': res}
```

5、完整流程构建

```
    def create_workflow(self):
        """创建工作流graph"""
        graph = StateGraph(StateNode)
        graph.add_node('检测用例前置依赖接口', self.check_dependency)
        graph.add_node('获取依赖接口', self.get_dependency_apis)
        graph.add_node('获取测试环境数据', self.get_test_env_data)
        graph.add_node('结构化用例生成', self.structure_case_generation)
        graph.add_node('验证用例的可用性', self.check_case)
        graph.add_node('判断用例是否可用', self.check_case_available)
        graph.add_node('标记为问题用例', self.mark_problem_case)
        graph.add_node('重新生成用例', self.re_generate_case)
        graph.add_node('保存用例', self.save_case)

        # 流程编排
        graph.add_conditional_edges(START, self.check_dependency, ['获取依赖接口',
'获取测试环境数据'])
        graph.add_edge('获取依赖接口', '获取测试环境数据')
        graph.add_edge('获取测试环境数据', '结构化用例生成')
        graph.add_edge('结构化用例生成', '验证用例的可用性')
        graph.add_conditional_edges('验证用例的可用性', self.check_case_available,
                                    ['保存用例', '标记为问题用例', '重新生成用例'])
        graph.add_edge('标记为问题用例', '保存用例')
        graph.add_edge('重新生成用例', '验证用例的可用性')
```

```
        graph.add_edge('保存用例', END)

        return graph.compile()
```

6、父Graph并行批量生成

```
# @Author  : 木森
# @weixin: python771
"""
设计一个父图，实现多个用例并行结构化处理

"""
from typing import TypedDict

from langgraph.graph import StateGraph
from langgraph.constants import START, END
from langgraph.types import Send

from workflows.strutured_case_generation import ApiCaseStructuredGeneratorFlow
from typing import Annotated, List
import operator


# 定义状态节点
class StateNode(TypedDict):
    base_cases: list  # 基础测试用例集
    project_name: str  # 项目名称
    api_info: dict  # 接口文档数据
    additional_info: str  # 用例生成的额外说明信息
    test_data: dict  # 执行环境的测试数据(测试账号什么的)

    task_base_case: str  # 并发生成用例的基础用例信息
    structured_cases: Annotated[List, operator.add]  # 最终输入的结构化的用例集


class StructuredCaseGeneratorFlow:
    """结构化用例批量并行生成的workflow"""

    @staticmethod
    def generation_task_split(state: StateNode):
        """生成的任务拆分"""
        base_cases = state['base_cases']
        # Send
        task_list = []
        for case_info in base_cases:
            task_list.append(
                Send('生成结构化用例', {
                    "task_base_case": case_info,
                    "api_info": state['api_info'],
                    "project_name": state['project_name'],
                    "additional_info": state.get('additional_info', ''),
                    "test_data": state.get('test_data', {})
                })
            )
        return task_list
```

```
    @staticmethod
    def structured_case_generation(state: StateNode):
        """结构化用例生成"""
        # 使用子图去生成单条测试执行
        graph = ApiCaseStructuredGeneratorFlow()
        flow = graph.create_workflow()
        graph_state = flow.invoke({"case_info": state['task_base_case'],
                                   "case_api": state['api_info'],
                                   "project": state['project_name'],
                                   "test_data": state.get('test_data', {}),
                                   "additional_info":
state.get('additional_info', ''),
                                   })
        case = graph_state['output_case']
        # 将生成的用例进行合并处理
        return {'structured_cases': [case]}

    @staticmethod
    def get_generation_cases(state: StateNode):
        """获取生成的所有用例"""
        print("==================生成完毕，完整用例如下=======================")
        print(state['structured_cases'])

    def create_workflow(self):
        """创建graph工作流"""
        graph = StateGraph(StateNode)
        graph.add_node('批量生成任务拆分', self.generation_task_split)
        graph.add_node('生成结构化用例', self.structured_case_generation)
        graph.add_node('获取生成结果', self.get_generation_cases)
        graph.add_conditional_edges(START, self.generation_task_split, ['生成结构
化用例'])
        graph.add_edge('生成结构化用例', '获取生成结果')
        graph.add_edge('获取生成结果', END)
        return graph.compile()
```

7、完整代码如下

1、单用例生成

```
# @Author  : 木森
# @weixin: python771
"""
单条用例生成的流程实现
"""
from langgraph.graph import StateGraph
from langgraph.constants import START, END
from typing import TypedDict, List
import traceback
from pydantic import BaseModel, Field
from langchain_core.output_parsers import JsonOutputParser
from agents.models import deepseek_v3_llm
from api_auto_test.common.base_cases import BaseTestCase
from api_auto_test.common.inspect_env import inspect_class_functions,
inspect_file_data
```

```python
from api_auto_test.main import run_test
from prompts.case_generator.re_generation_case import re_generate_case_prompt
from prompts.case_generator.structured_case_generator import output_format,
structured_case_prompt
from rag.data_storage.sql_storage import SQLDB


class StateNode(TypedDict):
    """定义节点的状态数据"""
    project: str  # 项目名称
    case_api: dict  # 用例接口
    case_info: dict  # 基础用例信息
    additional_info: str  # 额外的说明信息
    other_api: list  # 前置条件依赖的接口详情
    test_data: str  # 测试数据(放在测试环境里面)
    files_list: list  # 文件上传的用例可用的文件列表
    function_list: list  # 生成测试数据和加密处理的可用的函数列表
    output_case: dict  # 最终输出的用例
    test_report: dict  # 验证用例可用性得到的测试报告
    is_available: bool  # 用例是否可用
    generator_count: int  # 用例生成的次数
    review_state: str  # 用例预执行的评审结果


class CaseParser(BaseModel):
    """结构化用例数据提取类"""
    name: str = Field(..., description='用例名称')
    description: str = Field('无', description='用例描述')
    project: str = Field('', description='项目名称')
    module: str = Field('', description='所属模块')
    interface: str = Field('', description='接口名称')
    preconditions: List = Field(default_factory=list, description='前置条件')
    request: dict = Field(default_factory=dict, description='请求体')
    assertions: dict = Field(default_factory=dict, description='断言信息')


class ApiCaseStructuredGeneratorFlow:
    """接口用例结构化生成的工作流"""

    @staticmethod
    def check_dependency(state: StateNode):
        """检测用例前置依赖接口"""
        case_info = state.get("case_info")
        dependencies = case_info.get('dependencies', [])
        if dependencies:
            # 有前置条件和依赖的接口(检索出依赖接口详细信息)
            return '获取依赖接口'
        else:
            # 无前置条件,获取用例生成的可用环境数据
            return '获取测试环境数据'

    @staticmethod
    def get_dependency_apis(state: StateNode):
        """获取依赖接口"""
        sql = 'SELECT * from api_interface WHERE project=%s AND summary  IN %s'
        case_info = state.get("case_info")
        dependencies = case_info.get('dependencies', [])
        with SQLDB() as db:
```

```
            db.cursor.execute(sql, (state.get('project'), dependencies))
            other_api = db.cursor.fetchall()
            print("前置依赖接口数据如下：", other_api)
            return {'other_api': other_api}

    @staticmethod
    def get_test_env_data(state: StateNode):
        """获取测试环境数据"""
        # 获取当前的测试环境中提供的测试数据
        test_data = state['test_data']
        # 获取测试数据随机生成的可用函数
        functions = inspect_class_functions()
        # 获取文件上传参数的可用文件列表
        files = inspect_file_data()
        return {'test_data': test_data, 'function_list': functions,
'files_list': files}

    @staticmethod
    def structure_case_generation(state: StateNode):
        """结构化用例生成"""
        try:
            # 准备提示词
            # output_format
            # 准备数据解析器
            parser = JsonOutputParser(pydantic_schema=CaseParser)
            # 构建调用链
            chain = structured_case_prompt | deepseek_v3_llm | parser
            # 生成用例
            response = chain.invoke({
                "output_format": output_format,
                "case_info": state.get("case_info", ''),
                "case_api": state.get("case_api", ''),
                "other_api": state.get("other_api", ''),
                "test_data": state.get("test_data", ''),
                "function_list": state.get("function_list", ''),
                "files_list": state.get("files_list", ''),
                "additional_info": state.get("additional_info", '')

            })
            print("生成的测试用例如下：", response)
            return {'output_case': response}
        except Exception as e:
            print("生成用例失败：", e)
            return {'output_case': None}

    @staticmethod
    def check_case(state: StateNode):
        """
        验证用例的可用性和正确性
        可用性：最直接的方式：运行该用例看运行过程是否出现错误
        :param state:
        :return:
        """

        # 获取生成的用例
        case = state.get("output_case")
        if not case:
            return {"test_report": '用例生成失败', 'is_available': False,
```

```
                    'review_state': "错误"}
        try:
            # 对用例数据进行预执行验证可用性和正确性
            case = BaseTestCase(case)
            case.run()
        except AssertionError as e:
            # 提取异常溯源信息
            trac = traceback.format_exc()
            print("执行可用性验证出现断言错误，用例执行失败", e)
            generator_count = state.get('generator_count', 0)
            return {"test_report": trac, 'is_available': False,
'generator_count': generator_count + 1,
                    'review_state': "失败"
                    }
        except Exception as e:
            trac = traceback.format_exc()
            print("执行可用性验证出现异常，用例执行错误", e)
            generator_count = state.get('generator_count', 0)
            return {"test_report": trac, 'is_available': False,
'generator_count': generator_count + 1,
                    'review_state': "错误"}
        else:
            print("执行可用性验证通过")
            case = state.get("output_case")
            case['review_status'] = '通过'
            return {'is_available': True, 'output_case': case}

    # 判断用例生成的次数
    @staticmethod
    def check_case_available(state: StateNode):
        """校验用例是否可用的分支处理"""
        generator_count = state.get('generator_count', 0)
        if state['is_available']:
            return '保存用例'
        elif generator_count > 3:
            return '标记问题用例'
        else:
            return '重新生成用例'

    @staticmethod
    def save_case(state: StateNode):
        """保存用例"""
        case = state.get("output_case")
        print("用例生成流程执行完毕，开始保存用例", case)
        return {"output_case": case}

    @staticmethod
    def mark_problem_case(state: StateNode):
        """标记问题用例"""
        case = state.get("output_case")
        case['review_status'] = state.get('review_state', '')
        print("标记问题用例", case)
        return {'output_case': case}

    @staticmethod
    def re_generate_case(state: StateNode):
        """重新生成用例"""
        try:
```

```
            print("正在开始重新生成用例数据")
            parser = JsonOutputParser(pydantic_schema=CaseParser)
            # 构建调用链
            chain = re_generate_case_prompt | deepseek_v3_llm | parser
            # 重新生成用例
            res = chain.invoke({
                "output_format": output_format,
                "case_info": state.get("case_info", ''),
                "case_api": state.get("case_api", ''),
                "other_api": state.get("other_api", ''),
                "test_data": state.get("test_data", ''),
                "function_list": state.get("function_list", ''),
                "files_list": state.get("files_list", ''),
                "additional_info": state.get("additional_info", ''),
                "case_data": state.get("output_case", ''),
                "execution_error": state.get('test_report', '')
            })
            print("重新生成的测试用例如下：", res)
            return {'output_case': res}
        except Exception as e:
            print("生成用例错误：", e)
            return {'output_case': None}

    def create_workflow(self):
        """创建工作流graph"""
        graph = StateGraph(StateNode)
        graph.add_node('检测用例前置依赖接口', self.check_dependency)
        graph.add_node('获取依赖接口', self.get_dependency_apis)
        graph.add_node('获取测试环境数据', self.get_test_env_data)
        graph.add_node('结构化用例生成', self.structure_case_generation)
        graph.add_node('验证用例的可用性', self.check_case)
        graph.add_node('判断用例是否可用', self.structure_case_generation)
        graph.add_node('标记问题用例', self.mark_problem_case)
        graph.add_node('重新生成用例', self.re_generate_case)
        graph.add_node('保存用例', self.save_case)
        # 流程编排
        graph.add_conditional_edges(START, self.check_dependency, ['获取依赖接口',
'获取测试环境数据'])
        graph.add_edge('获取依赖接口', '获取测试环境数据')
        graph.add_edge('获取测试环境数据', '结构化用例生成')
        graph.add_edge('结构化用例生成', '验证用例的可用性')
        graph.add_conditional_edges('验证用例的可用性', self.check_case_available,
                                    ['保存用例', '重新生成用例', '标记问题用例'])

        graph.add_edge('标记问题用例', '保存用例')
        graph.add_edge('重新生成用例', '验证用例的可用性')
        graph.add_edge('保存用例', END)

        return graph.compile()


if __name__ == '__main__':
    case_info = {'name': '正常创建项目-参数正确',
                 'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/',
                           '请求体包含正确的name和user参数'],
                 'expected': ['HTTP状态码为201', '响应体包含id、name、user_id和
create_time字段', 'name值与请求参数一致'],
```

```
                 'dependencies': ['用户登录']}
    case_api = {'version': None, 'description': '创建项目', 'path':
'/api/pro/projects/', 'tag': None,
                'responses': '{"201": {"id": {"type": "integer", "description":
"项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id":
{"type": "integer", "description": "项目负责人"}, "create_time": {"type":
"string", "description": "创建时间"}}, "422": {"detail": {"type": "array",
"description": ""}}}',
                'summary': '创建项目', 'method': 'POST', 'module': None,
                'requestBody': '{"name": {"type": "string", "description": "项目
名称"}, "user": {"type": "integer", "description": "项目负责人"}}',
                'project': 'web测试平台', 'parameters': '[]', 'distance':
-3.5762786865234375e-07}
    project = 'web测试平台'

    app = ApiCaseStructuredGeneratorFlow().create_workflow()
    response = app.invoke({"case_info": case_info,
                           "case_api": case_api,
                           "project": project,
                           "test_data": {
                               "username": "python771",
                               "password": "musen12399",
                               "base_url": "http://124.222.144.182:8888"
                           }}, )
```

2、批量并行生成用例流程

```
# @Author  : 木森
# @weixin: python771
"""
设计一个父图，实现多个用例并行结构化处理

"""
import operator

from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated, List

from langgraph.types import Send

from workflows.strutured_case_generation import ApiCaseStructuredGeneratorFlow


# 假设StateNode已在主文件定义，这里仅作类型提示
class StateNode(TypedDict):
    base_cases: list  # 基础用例列表
    project_name: str  # 项目名称
    api_info: dict  # 接口数据
    additional_info: str  # 附加补充信息
    structured_cases: Annotated[List, operator.add]  # 结构化用例列表
    task_case: dict  # 并发用例信息


class StructuredCaseGeneratorGraph:
    """批量结构化生成api用例"""
```

```
    @staticmethod
    def case_flow_graph(state: StateNode):
        """使用子图批量生成用例"""
        flow = ApiCaseStructuredGeneratorFlow()
        graph = flow.create_workflow()
        case_ = graph.invoke({
            "case_info": state['task_case'],
            "api_info": state['api_info'],
            "project_name": state['project_name'],
            "additional_info": state['additional_info']
        })
        return {"structured_cases": [case_['output_case']]}

    # 遍历基础用例列表，并发调用生成用例的子图
    @staticmethod
    def generate_task_split(state: StateNode):
        """任务拆分子图并行生成"""
        base_cases = state['base_cases']
        return [Send('执行用例结构化生成任务子图', {
            'task_case': case_info,
            'api_info': state['api_info'],
            'project_name': state['project_name'],
            'additional_info': state['additional_info']
        }) for case_info in base_cases]

    # 用例存储
    @staticmethod
    def store_cases(state: StateNode):
        """用例存储"""
        print('用例生成完成，开始进行结构化存储')
        print(state['structured_cases'])

    def workflow(self):
        """构建流程图"""
        graph = StateGraph(StateNode)
        graph.add_node('生成任务拆分', self.generate_task_split)
        graph.add_node('执行用例结构化生成任务子图', self.case_flow_graph)
        graph.add_node('用例存储', self.store_cases)
        # 流程编排
        graph.add_conditional_edges(START, self.generate_task_split, ['执行用例结
构化生成任务子图'])
        graph.add_edge('执行用例结构化生成任务子图', '用例存储')
        graph.add_edge('用例存储', END)
        return graph.compile()


if __name__ == '__main__':
    cases = [
        {'name': '正常创建项目-所有必填参数正确',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，包含正确的name和user参数'],
         'expected': ['HTTP状态码为201', '响应体包含id、name、user_id、create_time字
段', '各字段数据类型符合接口文档定义'],
         'dependencies': ['用户登录']},
        {'name': '创建项目-缺少name参数',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，只包含user参数'],
```

```
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-缺少user参数',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，只包含name参数'],
         'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-name参数为空字符串',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，name参数为空字符串'],
         'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-name参数超过最大长度',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，name参数为超过限制长度的字符串'],
         'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-user参数为非法用户ID',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，user参数为不存在的用户ID'],
         'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-user参数为字符串类型',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，user参数为字符串类型'],
         'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-未授权访问',
         'steps': ['不提供token信息', '发送POST请求到/api/pro/projects/，包含正确的请
求参数'],
         'expected': ['HTTP状态码为401'], 'dependencies': []},
        {'name': '创建项目-name参数为null',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，name参数为null'],
         'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
        {'name': '创建项目-使用过期的token',
         'steps': ['使用过期的token', '发送POST请求到/api/pro/projects/，包含正确的请
求参数'],
         'expected': ['HTTP状态码为401'], 'dependencies': ['用户登录']},
        {'name': '创建项目-使用GET方法请求',
         'steps': ['调用用户登录接口获取有效token', '发送GET请求到/api/pro/projects/，
包含正确的请求参数'],
         'expected': ['HTTP状态码为405（不允许的方法）', '响应体包含方法不允许的错误信
息'], 'dependencies': ['用户登录']},
        {'name': '创建项目-添加未定义的参数', 'steps': ['调用用户登录接口获取有效token',
                                                        '发送POST请求
到/api/pro/projects/，包含name、user参数外，额外增加未定义的参数（如：extra_field）'],
         'expected': ['HTTP状态码为201（成功处理）', '响应体仅包含定义的字段
（id/name/user_id/create_time）',
                      '未定义的参数被忽略'], 'dependencies': ['用户登录']},
        {'name': '正常创建项目-所有必填参数正确',
         'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，包含正确的name和user参数'],
         'expected': ['HTTP状态码为201', '响应体包含id、name、user_id、create_time字
段', '各字段数据类型符合接口文档定义'],
         'dependencies': ['用户登录']},
        {'name': '创建项目-缺少name参数',
```

```
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，只包含user参数'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-缺少user参数',
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，只包含name参数'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-name参数为空字符串',
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，name参数为空字符串'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-name参数超过最大长度',
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，name参数为超过限制长度的字符串'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-user参数为非法用户ID',
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，user参数为不存在的用户ID'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-user参数为字符串类型',
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，user参数为字符串类型'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-未授权访问',
        'steps': ['不提供token信息', '发送POST请求到/api/pro/projects/，包含正确的请
求参数'],
        'expected': ['HTTP状态码为401'], 'dependencies': []},
       {'name': '创建项目-name参数为null',
        'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/，name参数为null'],
        'expected': ['HTTP状态码为422', '响应体包含错误详情信息'], 'dependencies':
['用户登录']},
       {'name': '创建项目-使用过期的token',
        'steps': ['使用过期的token', '发送POST请求到/api/pro/projects/，包含正确的请
求参数'],
        'expected': ['HTTP状态码为401'], 'dependencies': ['用户登录']},
       {'name': '创建项目-使用GET方法请求',
        'steps': ['调用用户登录接口获取有效token', '发送GET请求到/api/pro/projects/，
包含正确的请求参数'],
        'expected': ['HTTP状态码为405（不允许的方法）', '响应体包含方法不允许的错误信
息'], 'dependencies': ['用户登录']},
       {'name': '创建项目-添加未定义的参数', 'steps': ['调用用户登录接口获取有效token',
                                                        '发送POST请求
到/api/pro/projects/，包含name、user参数外，额外增加未定义的参数（如：extra_field）'],
        'expected': ['HTTP状态码为201（成功处理）', '响应体仅包含定义的字段
（id/name/user_id/create_time）',
                     '未定义的参数被忽略'], 'dependencies': ['用户登录']}

    ]
    work = StructuredCaseGeneratorGraph()
    app = work.workflow()
    response = app.invoke({"base_cases": cases,
```

```
                            "api_info": {'version': None, 'description': '创建项
目', 'path': '/api/pro/projects/',
                                         'tag': None,
                                         'responses': '{"201": {"id": {"type":
"integer", "description": "项目id"}, "name": {"type": "string", "description":
"项目名称"}, "user_id": {"type": "integer", "description": "项目负责人"},
"create_time": {"type": "string", "description": "创建时间"}}, "422": {"detail":
{"type": "array", "description": ""}}}',
                                         'summary': '创建项目', 'method': 'POST',
'module': None,
                                         'requestBody': '{"name": {"type":
"string", "description": "项目名称"}, "user": {"type": "integer", "description":
"项目负责人"}}',
                                         'project': 'web测试平台', 'parameters':
'[]',
                                         "security": [{"OAuth2PasswordBearer":
[]}]
                                         },
                            "project_name": "web测试平台",
                            "additional_info": """
                            接口鉴权使用OAuth2PasswordBearer认证，请求头中传递
Authorization: Bearer {token},
                            创建接口参数的name字段随机生成，user字段为登录之后提取的用户id

                            """

                            })

 print("*************************************************************************"
)
    print(response['structured_cases'])
```